

Food Ordering System

💡 Problem Statement:

To design and implement a Food Ordering System using Python that allows users to select food items from different categories (like pizza, drinks, burgers, desserts), add items with desired quantities to the cart, apply discount coupons, calculate total payable amount with tax (GST), preview the order summary, and finally save the bill as a text file.

The system should provide an attractive user interface using Tkinter, along with additional features such as dice animation, sound toggle, and real-time total calculation.

📁 Concepts Used:

Concept	Purpose
Tkinter (UI)	For designing the interactive GUI (buttons, labels, dropdowns, billing area)
Functions	For modular operations like adding items, applying coupons, calculating totals, saving bills
Dictionaries	To store menu items and prices under categories
Lists	To maintain the dynamic cart with multiple food items
String Formatting	For displaying formatted bills and summaries
File Handling	Saving the final bill into a text file
Modules	Use of pre-built modules (Tkinter, datetime, winsound)
Conditional Statements	Validating coupons, checking empty cart, verifying quantity
Loops	Iterating over cart items to display or compute totals

📄 Short Description of Working:

1. The system loads multiple food categories such as Pizzas, Burgers, Drinks, and Desserts.
2. The user selects an item and quantity using dropdown menus and input boxes.
3. The selected item is added to the cart, where it can accumulate multiple entries.
4. The order total is updated automatically, including:
 - ✓ GST of 12%
 - ✓ Discount coupons (e.g., SAVE50, NEW10, FEST20)
5. The user may apply or remove a coupon anytime before checkout.
6. A preview of the final bill is shown with all details (item name, qty, item total, tax, discount, grand total).

7. On confirming, the bill is saved into a text file using an auto-increment bill number (e.g., Order_Bill_No_003.txt).
8. The system provides options for:
 - Restarting the order
 - Sound ON/OFF control for interactive ordering
 - User-friendly, colorful UI with real-time updates

◆ **Logic Flow:**

1. Start
2. Load categories, price list, and initialize cart.
3. Wait for user actions (Add Item, Apply Coupon, Toggle Sound).
4. When an item is selected → store item & quantity into cart.
5. Update subtotal, discount, GST, and grand total.
6. If coupon button clicked → validate and update discount.
7. On checkout:
 - Generate bill
 - Save to text file
 - Display confirmation
8. End / Restart.

 foodOrderingSystem.py

```

main.py
# FOOD ORDERING SYSTEM (CONSOLE PROJECT)

import datetime

# ----- Configuration -----

TAX_RATE = 0.12 # 12% GST

MENU = {
    "Pizzas": {
        1: ("Margherita", 250),
        2: ("Farmhouse", 320),
        3: ("Pepperoni", 350),
    },
    "Burgers": {
        4: ("Veg Burger", 150),
        5: ("Cheese Burger", 180),
        6: ("Chicken Burger", 220),
    },
    "Drinks": {
        7: ("Coke", 60),
        8: ("Lemonade", 80),
        9: ("Cold Coffee", 120),
    },
    "Desserts": {
        10: ("Brownie", 140),
        11: ("Ice Cream", 90),
        12: ("Cheesecake", 180),
    }
}

COUPONS = {
    "SAVE50": {"type": "flat", "value": 50}, # flat ₹50 off
    "NEW10": {"type": "percent", "value": 10}, # 10% off
    "FEST20": {"type": "percent", "value": 20} # 20% off
}

# ----- Helper Functions -----

def print_header(): # 1 usage
    print("\n" + "=" * 40)
    print("          FOOD ORDERING SYSTEM          ")
    print("=" * 40)

def display_menu(): # 2 usages
    print("\n==== MENU ====")
    for category, items in MENU.items():
        print(f"\n-- {category} --")
        for code, (name, price) in items.items():
            print(f"{code}. {name} - ₹{price}")

```

```

def find_item_by_code(code): 1 usage
    """Return (name, price) given item code, else None."""
    for items in MENU.values():
        if code in items:
            return items[code]
    return None

def display_cart(cart): 2 usages
    if not cart:
        print("\nYour cart is empty.")
        return

    print("\n===== YOUR CART =====")
    print("{:<4} {:<20} {:<8} {:<8}".format(*args: "No.", "Item", "Qty", "Total"))
    print("-" * 40)
    for idx, item in enumerate(cart, start=1):
        name = item["name"]
        qty = item["quantity"]
        price = item["price"]
        lineTotal = qty * price
        print(f"{idx:<4} {name:<20} {qty:<8} ₹{lineTotal:<8}")
    print("-" * 40)

def calculate_subtotal(cart): 1 usage
    return sum(item["price"] * item["quantity"] for item in cart)

def apply_coupon(subtotal, code): 1 usage
    """Return (discount_amount, coupon_used)."""
    if not code:
        return 0, None

    code = code.upper()
    if code not in COUPONS:
        print("Invalid coupon code. No discount applied.")
        return 0, None

    coupon = COUPONS[code]
    if coupon["type"] == "flat":
        discount = coupon["value"]
    else: # percent
        discount = (coupon["value"] / 100) * subtotal

    # Avoid negative totals
    discount = min(discount, subtotal)

    print(f"Coupon '{code}' applied. Discount: ₹{int(discount)}")
    return int(discount), code

def save_bill_to_file(cart, subtotal, taxAmount, discount, total, coupon_code): 1 usage
    now = datetime.datetime.now()
    filename = f"bill_{now.strftime('%Y%m%d_%H%M%S')}.txt"

    with open(filename, "w", encoding="utf-8") as f:
        f.write("===== FOOD ORDERING SYSTEM - BILL =====\n")
        f.write(f>Date & Time: {now}\n")
        f.write("-" * 40 + "\n")

```

```

f.write("{:<4} {:<20} {:<8} {:<8}\n".format(*args: "No.", "Item", "Qty", "Total"))
f.write("-" * 40 + "\n")

for idx, item in enumerate(cart, start=1):
    name = item["name"]
    qty = item["quantity"]
    price = item["price"]
    lineTotal = qty * price
    f.write(f"{idx:<4} {name:<20} {qty:<8} ₹{lineTotal:<8}\n")

f.write("-" * 40 + "\n")
f.write(f"Subtotal: ₹{subtotal}\n")
f.write(f"GST @ {int(TAX_RATE * 100)}%: ₹{taxAmount}\n")
if coupon_code:
    f.write(f"Coupon ({coupon_code}) Discount: -₹{discount}\n")
f.write(f"Grand Total: ₹{total}\n")
f.write("Thank you for ordering with us!\n")

print(f"\nBill saved as '{filename}'")

# ----- Main Flow -----

def main(): 1 usage
    cart = []
    print_header()

    while True:
        print("\n1. View Menu")
        print("2. Add Item to Cart")
        print("3. View Cart")
        print("4. Checkout")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ").strip()

        if choice == "1":
            display_menu()

        elif choice == "2":
            display_menu()
            try:
                code = int(input("\nEnter item code to add: "))
                item = find_item_by_code(code)
                if item is None:
                    print("Invalid item code.")
                    continue

                name, price = item
                qty = int(input(f"Enter quantity for {name}: "))
                if qty <= 0:
                    print("Quantity must be positive.")
                    continue

```

```

        # Check if item already in cart -> update quantity
        for cItem in cart:
            if cItem["name"] == name:
                cItem["quantity"] += qty
                break
            else:
                cart.append({"name": name, "price": price, "quantity": qty})

        print(f"{qty} x {name} added to cart.")

    except ValueError:
        print("Please enter valid numeric values.")

elif choice == "3":
    display_cart(cart)

elif choice == "4":
    if not cart:
        print("\nYour cart is empty. Add items before checkout.")
        continue

    # Show order summary
    display_cart(cart)
    subtotal = calculate_subtotal(cart)
    taxAmount = int(subtotal * TAX_RATE)
    print(f"\nSubtotal: ₹{subtotal}")
    print(f"GST @ {int(TAX_RATE * 100)}%: ₹{taxAmount}")

    # Apply coupon
    couponInput = input("Enter coupon code (or press Enter to skip): ").strip()
    discount, couponUsed = apply_coupon(subtotal, couponInput)

    total = subtotal + taxAmount - discount
    total = int(total)

    print("\n==== ORDER SUMMARY =====")
    print(f"Subtotal: ₹{subtotal}")
    print(f"GST: ₹{taxAmount}")
    if couponUsed:
        print(f"Coupon ({couponUsed}) Discount: -₹{discount}")
    print(f"Grand Total: ₹{total}")

    confirm = input("\nConfirm order? (y/n): ").strip().lower()
    if confirm == "y":
        print("\nOrder confirmed! 🎉")
        saveChoice = input("Do you want to save the bill to a file? (y/n): ").strip().lower()
        if saveChoice == "y":
            save_bill_to_file(cart, subtotal, taxAmount, discount, total, couponUsed)
            print("Thank you for ordering. Visit again!")
            break
        else:
            print("Order cancelled. You can continue modifying your cart.")

    elif choice == "5":
        print("\nExiting... Thank you!")
        break

    else:
        print("Invalid choice. Please select between 1-5.")

```



 Output Screenshot (Representation):

```

↓
☰
≡
🖨
🗑
=====
FOOD ORDERING SYSTEM
=====

1. View Menu
2. Add Item to Cart
3. View Cart
4. Checkout
5. Exit
Enter your choice (1-5): 1

==== MENU ====

-- Pizzas --
1. Margherita - ₹250
2. Farmhouse - ₹320
3. Pepperoni - ₹350

-- Burgers --
4. Veg Burger - ₹150
5. Cheese Burger - ₹180
6. Chicken Burger - ₹220

-- Drinks --
7. Coke - ₹60
8. Lemonade - ₹80
9. Cold Coffee - ₹120

-- Desserts --
10. Brownie - ₹140
11. Ice Cream - ₹90
12. Cheesecake - ₹180

1. View Menu
2. Add Item to Cart
3. View Cart
4. Checkout
5. Exit
Enter your choice (1-5): |
```

Food Ordering System

💡 Problem Statement:

To design and implement a **Personal Notebook / Reminder App** using Python that allows users to write and store daily notes or reminders with an automatic timestamp. The application should enable users to **add, view, search, edit, and delete notes** through a user-friendly interface. All notes must be stored permanently using **file handling**, and the system should update the saved data whenever the user modifies it.

📚 Concepts Used:

Concept	Purpose
File Handling	To permanently store notes into a text file and retrieve them on startup
Functions	To modularize features like add, search, update, delete, and save
Strings & Searching	Used to search keywords inside saved notes
Dictionaries	To store note data (timestamp + message) in structured form
Lists	To maintain multiple notes in memory
Date & Time (datetime)	To add automatic timestamps for every note
Tkinter GUI	For creating an interactive and user-friendly interface
Conditional Statements	For validating user actions such as empty notes, invalid selection, etc.
Looping Structures	To display and refresh multiple notes in the UI

📄 Short Description of Working:

1. The system loads existing notes from a text file at startup using file handling.
2. The user writes a note in the input area and clicks **Add Note**.
3. The note gets automatically stamped with the current **date & time** and is stored in the file.
4. Users can view all notes in a scrollable table view.
5. A **Search feature** allows finding notes using keywords (e.g., “meeting”, “assignment”, etc.)
6. Users may select any note to **edit** or **delete** using dedicated buttons.
7. When an update or delete action occurs, the system overwrites the text file with new data.
8. All functionalities refresh dynamically without restarting the program.

◆ **Logic Flow:**

1. **Start Application**
2. Load notes from notes.txt
3. Wait for user action:
 - **Add Note**
 - **Search**
 - **Edit**
 - **Delete**
 - **Show All**
4. According to user action:
 - Add → Append new note + timestamp → Save file
 - Search → Filter notes by keyword
 - Edit/Delete → Select note from list → Modify or remove → Save file
5. Refresh notes view in UI
6. Exit if user closes window

 remainderNotebook.py

```

● ● ● main.py
# PERSONAL NOTEBOOK / REMINDER APP (Console Project)

import os
import datetime

NOTES_FILE = "notes.txt"

# ----- Data Handling Functions -----

def load_notes(): 1 usage
    """Load notes from file into a list of dicts: [{'date':..., 'message':...}, ...]"""
    notes = []
    if not os.path.exists(NOTES_FILE):
        return notes

    with open(NOTES_FILE, "r", encoding="utf-8") as f:
        lines = [line.rstrip("\n") for line in f]

    i = 0
    while i < len(lines):
        if lines[i].strip() == "----- NOTE -----":
            # Expect: Date line, Message line, separator
            dateLine = lines[i + 1] if i + 1 < len(lines) else ""
            msgLine = lines[i + 2] if i + 2 < len(lines) else ""
            # Move index to after block
            i = i + 4 # skip separator line too

            if dateLine.startswith("Date:") and msgLine.startswith("Message:"):
                dateText = dateLine.replace(__old: "Date:", __new: "", __count: 1).strip()
                messageText = msgLine.replace(__old: "Message:", __new: "", __count: 1).strip()
                notes.append({"date": dateText, "message": messageText})
            else:
                i += 1

    return notes

def save_notes(notes): 3 usages
    """Overwrite file with all notes in the block format."""
    with open(NOTES_FILE, "w", encoding="utf-8") as f:
        for note in notes:
            f.write("----- NOTE -----\\n")
            f.write(f"Date: {note['date']}\\n")
            f.write(f"Message: {note['message']}\\n")
            f.write("-----\\n")

```

```
# ----- Core Features -----

def add_note(notes): 1 usage
    print("\n📝 ADD NEW NOTE")
    text = input("Enter your note/reminder: ").strip()
    if not text:
        print("Note cannot be empty.")
        return

    now = datetime.datetime.now().strftime("%Y-%m-%d %H:%M")
    notes.append({"date": now, "message": text})
    save_notes(notes)
    print("✅ Note added successfully with timestamp:", now)

def view_notes(notes): 1 usage
    print("\n📋 ALL NOTES")
    if not notes:
        print("No notes found yet. Start by adding one!")
        return

    for idx, note in enumerate(notes, start=1):
        print("-" * 40)
        print(f"# {idx} 📅 {note['date']}")
        print(f"📄 {note['message']}")
    print("-" * 40)

def search_notes(notes): 3 usages
    print("\n🔍 SEARCH NOTES")
    keyword = input("Enter keyword to search: ").strip().lower()
    if not keyword:
        print("Keyword cannot be empty.")
        return []

    results = []
    for i, note in enumerate(notes):
        if keyword in note["message"].lower() or keyword in note["date"].lower():
            results.append((i, note))

    if not results:
        print("No notes matched your search.")
    else:
        print(f"\nFound {len(results)} matching note(s):")
        for idx, (original_index, note) in enumerate(results, start=1):
            print("-" * 40)
            print(f"Result {idx} (Note #{original_index + 1})")
            print(f"📅 {note['date']}")
            print(f"📄 {note['message']}")
        print("-" * 40)

    return results
```

```

def choose_from_search(results): 2 usages
    """Let user pick one note from search results. Returns original index or None."""
    if not results:
        return None

    try:
        choice = int(input("Enter result number to select (0 to cancel): "))
    except ValueError:
        print("Invalid input.")
        return None

    if choice == 0:
        return None

    if 1 <= choice <= len(results):
        originalIndex, _ = results[choice - 1]
        return originalIndex
    else:
        print("Invalid choice.")
        return None

def update_note(notes): 1 usage
    print("\n✏ UPDATE NOTE (search-based)")
    results = search_notes(notes)
    index = choose_from_search(results)
    if index is None:
        return

    old_note = notes[index]
    print("\nCurrent note:")
    print(f"📅 {old_note['date']}")
    print(f"📄 {old_note['message']}")

    new_text = input("\nEnter updated note text (leave blank to cancel): ").strip()
    if not new_text:
        print("Update cancelled.")
        return

    # Option: keep original date or update date. We'll update timestamp to show last edit.
    now = datetime.datetime.now().strftime("%Y-%m-%d %H:%M")
    notes[index] = {"date": now, "message": new_text}
    save_notes(notes)
    print("✅ Note updated successfully.")

def delete_note(notes): 1 usage
    print("\n🗑 DELETE NOTE (search-based)")
    results = search_notes(notes)
    index = choose_from_search(results)
    if index is None:
        return

```

```

    note = notes[index]
    print("\nYou are about to delete this note:")
    print(f"📅 {note['date']}")
    print(f"📄 {note['message']}")

    confirm = input("Are you sure? (y/n): ").strip().lower()
    if confirm == "y":
        notes.pop(index)
        save_notes(notes)
        print("✅ Note deleted.")
    else:
        print("Deletion cancelled.")

# ----- Menu / Main Loop -----

def print_menu(): 1 usage
    print("\n===== PERSONAL NOTEBOOK =====")
    print("1. 📝 Add New Note")
    print("2. 📖 View All Notes")
    print("3. 🔍 Search Notes")
    print("4. ✏️ Update Note (via search)")
    print("5. 🗑️ Delete Note (via search)")
    print("6. 🚪 Exit")
    print("=====")

def main(): 1 usage
    notes = load_notes()
    print("🌈 Welcome to Personal Notebook / Reminder App")
    print(f"Using file: {NOTES_FILE}")

    while True:
        print_menu()
        choice = input("Enter your choice (1-6): ").strip()

        if choice == "1":
            add_note(notes)
        elif choice == "2":
            view_notes(notes)
        elif choice == "3":
            search_notes(notes)
        elif choice == "4":
            update_note(notes)
        elif choice == "5":
            delete_note(notes)
        elif choice == "6":
            print("👋 Exiting Personal Notebook. Goodbye!")
            break
        else:
            print("Invalid choice. Please select between 1 and 6.")

```

Output Screenshot (Representation):

```

Welcome to Personal Notebook / Reminder App
Using file: notes.txt

===== PERSONAL NOTEBOOK =====
1. 📝 Add New Note
2. 📅 View All Notes
3. 🔍 Search Notes
4. ✏️ Update Note (via search)
5. 🗑️ Delete Note (via search)
6. 🚪 Exit
=====
Enter your choice (1-6): 1

📝 ADD NEW NOTE
Enter your note/reminder: Buy Fruits at Evening Walk
✅ Note added successfully with timestamp: 2025-12-06 16:27

===== PERSONAL NOTEBOOK =====
1. 📝 Add New Note
2. 📅 View All Notes
3. 🔍 Search Notes
4. ✏️ Update Note (via search)
5. 🗑️ Delete Note (via search)
6. 🚪 Exit
=====
Enter your choice (1-6): 2

📅 ALL NOTES
-----
#1 📅 2025-12-06 15:16
  ☐ Buy Fruits
-----
#2 📅 2025-12-06 16:27
  ☐ Buy Fruits at Evening Walk
-----

```